Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Bhakti Ganapati Revankar **1BM25EC405**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

INDEX:

Sl.No.	Title
1	Arithmetic logic unit
2	Comparator
3	4-Bit parity generator and checker
4	4-Bit barrel shifter
5	Timer module
6	Ring counter
7	Johnson counter
8	Synchronous and asynchronous counters
9	Shift register
10	Divide-by-N counter
11	Mealy and moore FSMs
12	Random sequence generator

Software Tool Used: EDA Playground

EDA Playground is a web-based Electronic Design Automation (EDA) tool used to write, compile, simulate, and verify digital circuit designs using Hardware Description Languages (HDLs) such as Verilog and SystemVerilog.

For this experiment, EDA Playground with the Icarus Verilog simulator is used to implement and test various digital circuits. The design code is written in the design editor, while the corresponding testbench is written separately to provide input signals and verify the outputs.

The Icarus Verilog simulator compiles and executes the Verilog code to check the functional behavior of the designed circuits. The simulation output is stored in a Value Change Dump (VCD) file, which can be viewed using EPWave, a waveform viewer integrated with EDA Playground. EPWave helps visualize changes in signals such as clock, reset, input, output, and internal states over time.

EDA Playground is useful for digital electronics and VLSI experiments because it does not require local software installation, supports quick testing and debugging, and allows students to observe and verify circuit behavior through simulation.

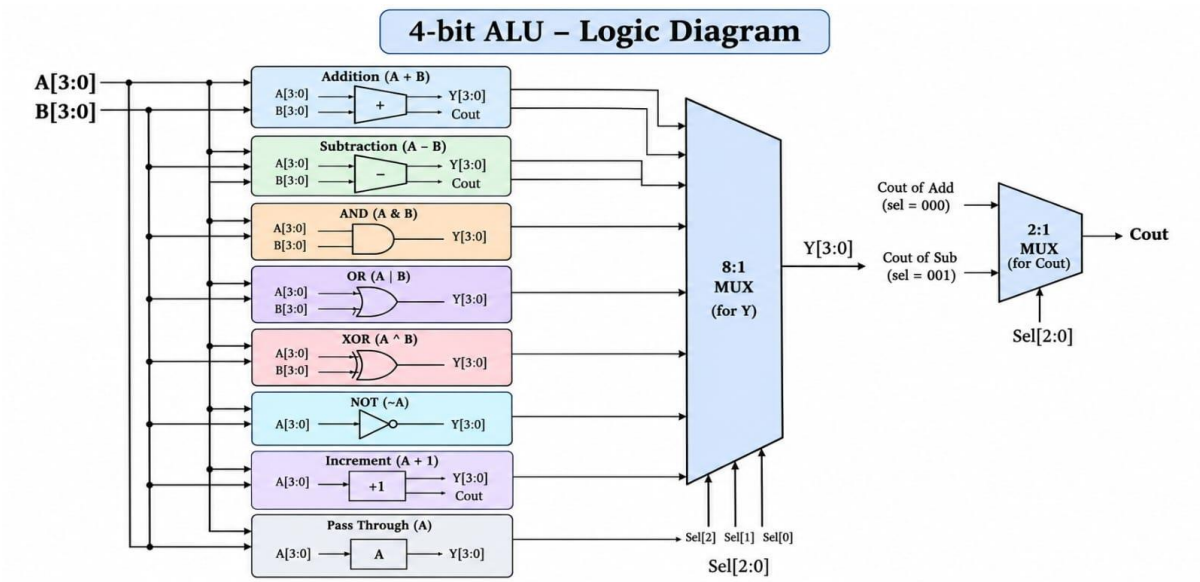
Tools used:

1. EDA Playground: Online platform for writing and running Verilog programs.
2. Icarus Verilog: Simulator used to compile and execute the Verilog designs.
3. EPWave: Waveform viewer used to analyze simulation results.

Experiment 1: Arithmetic Logic Unit (ALU)

Aim: To design a 4-bit ALU that performs arithmetic and logical operations based on a select signal.

Logic diagram:



Code:

```
// Code your testbench here
module tb_alu;
    reg [3:0] A, B;
    reg [2:0] Sel;
    wire [3:0] Y;
    wire Cout;

    alu_4bit dut (
        .A(A),
        .B(B),
        .Sel(Sel),
        .Y(Y),
        .Cout(Cout)
    );

    initial begin
        $monitor("Time=%0t A=%d B=%d Sel=%b Y=%d Cout=%b",
                 $time, A, B, Sel, Y, Cout);
        $dumpfile("waveform.vcd");
        $dumpvars(0, tb_alu);
        A = 4'd7;
        B = 4'd3;
        Sel = 3'b000; #10;
        Sel = 3'b001; #10;
        Sel = 3'b010; #10;
        Sel = 3'b011; #10;
        Sel = 3'b100; #10;
        Sel = 3'b101; #10;
        Sel = 3'b110; #10;
        Sel = 3'b111; #10;

        $finish;
    end
endmodule
```

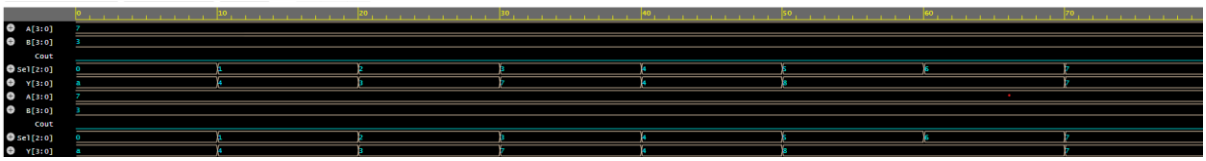
```
// Code your design here

module alu_4bit(
    input [3:0] A, B,
    input [2:0] Sel,
    output reg [3:0] Y,
    output reg Cout
);
always @(*) begin
    Y = 0;
    Cout = 0;

    case (Sel)
        3'b000: {Cout,Y} = {1'b0,A} + {1'b0,B};
        3'b001: Y = A - B;
        3'b010: Y = A & B;
        3'b011: Y = A | B;
        3'b100: Y = A ^ B;
        3'b101: Y = ~A;
        3'b110: {Cout,Y} = {1'b0,A} + 5'b00001;
        3'b111: Y = A;
        default: begin
            Y = 0;
            Cout = 0;
        end
    endcase
end

endmodule
```

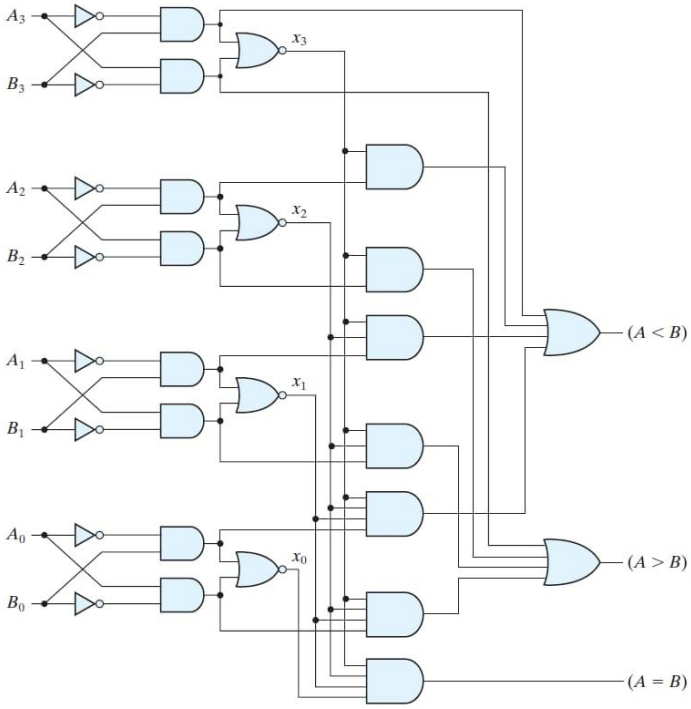
Output:



Experiment 2: 4-bit Comparator

Aim: To compare two 4-bit binary numbers and determine whether A is greater than, less than, or equal to B.

Logic diagram:



Code:

```
// Code your testbench here
module comparator_tb;

reg [3:0] A, B;
wire GT, LT, EQ;

comparator dut(
    .A(A),
    .B(B),
    .GT(GT),
    .LT(LT),
    .EQ(EQ)
);

initial begin
    $monitor("Time=%0t A=%d B=%d GT=%b LT=%b EQ=%b",
             $time, A, B, GT, LT, EQ);

    $dumpfile("waveform.vcd");
    $dumpvars(0, comparator_tb);

    A = 7; B = 3;  #10;
    A = 2; B = 5;  #10;
    A = 4; B = 4;  #10;
    A = 0; B = 15; #10;

    $finish;
end

endmodule
```

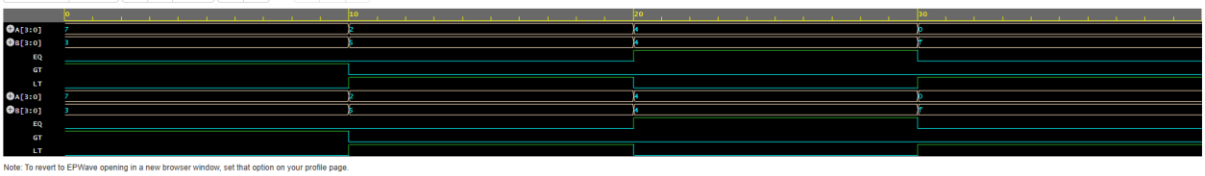
```
// Code your design here
module comparator(
    input [3:0] A,
    input [3:0] B,
    output reg GT,
    output reg LT,
    output reg EQ
);

always @(*) begin
    GT = 0;
    LT = 0;
    EQ = 0;

    if (A > B)
        GT = 1;
    else if (A < B)
        LT = 1;
    else
        EQ = 1;
end

endmodule
```

Output:

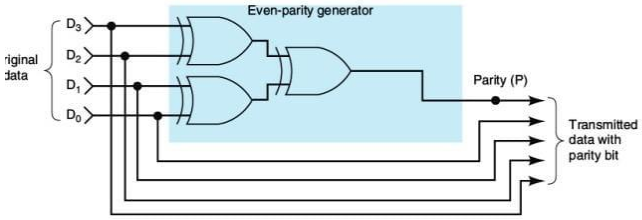


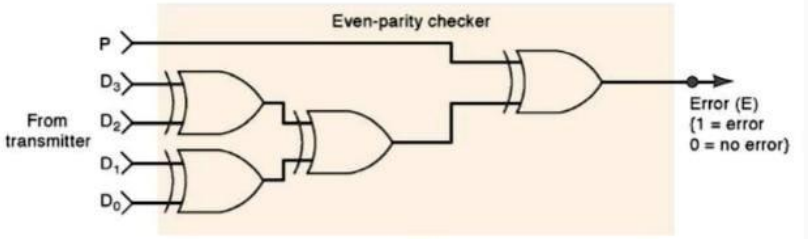
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

Experiment 3: 4-bit Parity Generator and Checker

Aim: To generate an even-parity bit and check the parity of received data.

Logic diagram:





Code:

```
// Code your testbench here

module tb_parity;

reg [3:0] data;
reg received_parity;
wire even_parity, error;

parity dut(data, received_parity, even_parity, error);

initial begin
    $monitor("Time=%0t data=%b received_parity=%b even_parity=%b error=%b",
             $time, data, received_parity, even_parity, error);

    $dumpfile("waveform.vcd");
    $dumpvars(0, tb_parity);

    data = 4'b1010;
    received_parity = 0; #10;
    received_parity = 1; #10;

    data = 4'b1110;
    received_parity = 0; #10;
    received_parity = 1; #10;

    $finish;
end

endmodule
```

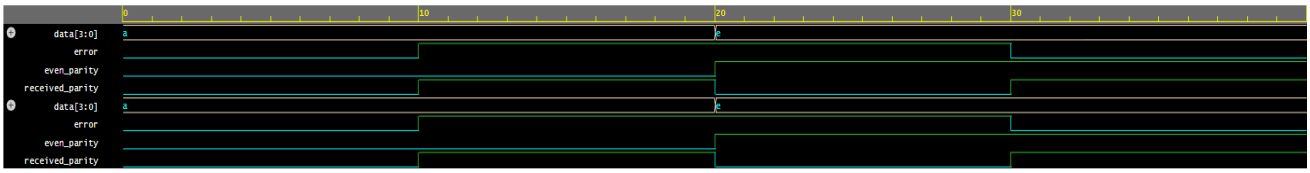
```
// Code your design here

module parity(
    input [3:0] data,
    input received_parity,
    output even_parity,
    output error
);

assign even_parity = ^data;
assign error = ^{data, received_parity};

endmodule
```

Output:

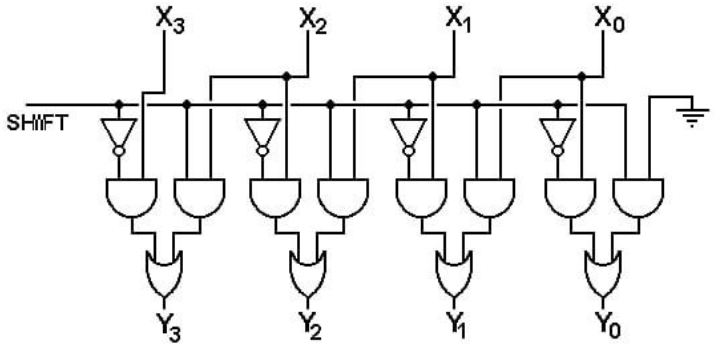


Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

Experiment 4: 4-bit Barrel Shifter

Aim: To shift a 4-bit input left or right by a selected number of positions.

Logic diagram:



Code:

```
// Code your testbench here
module tb_barrel_shifter;
reg [3:0] data;
reg [1:0] shift;
reg dir;
wire [3:0] result;
barrel_shifter dut (
    .data(data),
    .shift(shift),
    .dir(dir),
    .result(result)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_barrel_shifter);
    data = 4'b1011;
    dir = 0;
    shift = 0;
    #10;
    shift = 1;
    #10;
    shift = 2;
    #10;
    dir = 1;
    shift = 1;
    #10;
    shift = 2;
    #10;
    $finish;
end
endmodule
```

```
// Code your design here

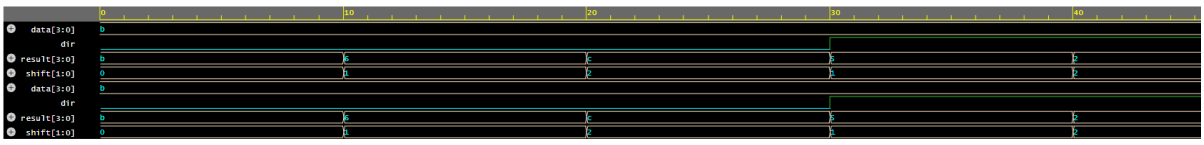

module barrel_shifter(
    input [3:0] data,
    input [1:0] shift,
    input dir,
    output reg [3:0] result
);

always @(*) begin
    if (dir == 1'b0)
        result = data << shift;
    else
        result = data >> shift;
end

endmodule

```

Output:

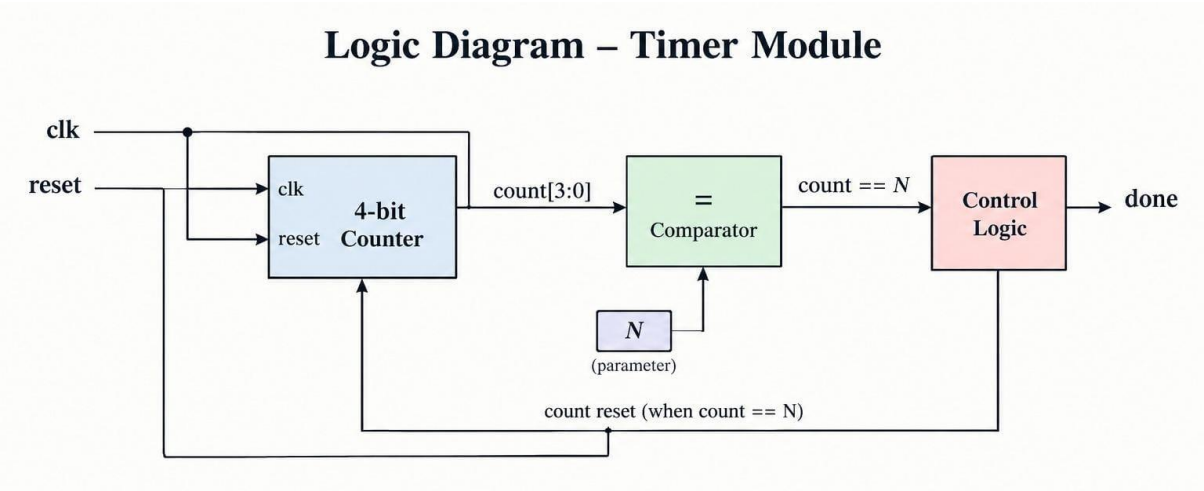


Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

Experiment 5: Timer Module

Aim: To design a timer that counts clock cycles and generates a signal when the count reaches a specified value.

Logic diagram:



Code:

```
// Code your testbench here

module tb_timer;

reg clk, reset;
wire done;

timer #(.N(9)) dut (
    .clk(clk),
    .reset(reset),
    .done(done)
);

always #5 clk = ~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_timer);

    clk = 0;
    reset = 1;

    #12 reset = 0;

    #150;
    $finish;
end

endmodule
```

```
// Code your design here

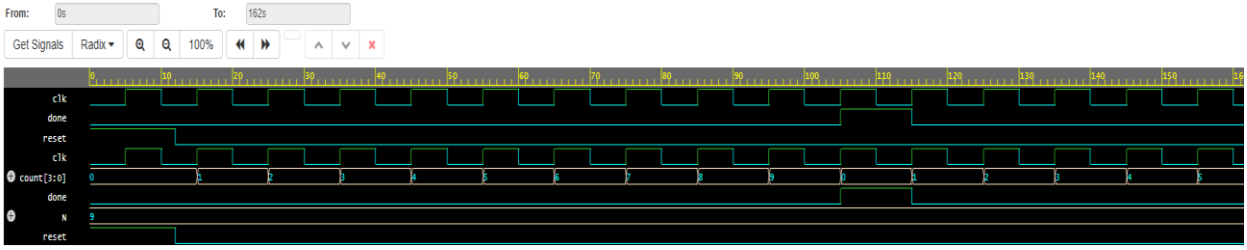

module timer #(
    parameter N = 9
)(
    input clk,
    input reset,
    output reg done
);

reg [3:0] count;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        count <= 0;
        done <= 0;
    end
    else if (count == N) begin
        count <= 0;
        done <= 1;
    end
    else begin
        count <= count + 1;
        done <= 0;
    end
end

endmodule
```

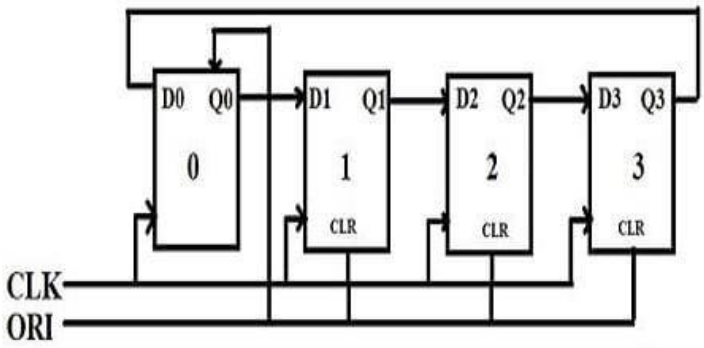
Output:



Experiment 6: Ring Counter

Aim: To design a 4-bit ring counter that circulates a single logic 1 through its flip-flops.

Logic diagram:



Code:

```
// Code your testbench here


module tb_ring_counter;

reg clk, reset;
wire [3:0] Q;

ring_counter dut(
    .clk(clk),
    .reset(reset),
    .Q(Q)
);

always #5 clk = ~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_ring_counter);

    clk = 0;
    reset = 1;
    #10;

    reset = 0;
    repeat (8) #10;

    $finish;
end

endmodule
```

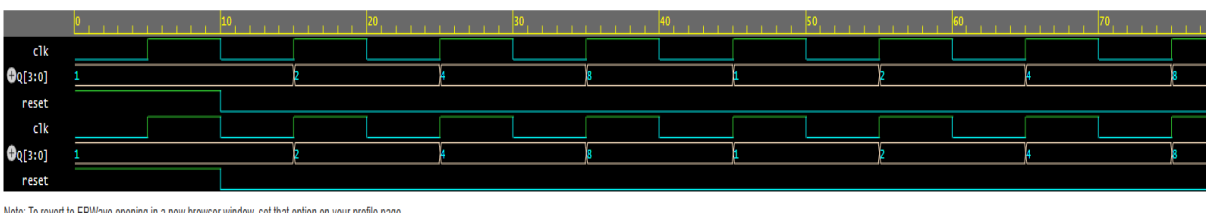
```
// Code your design here


module ring_counter(
    input clk,
    input reset,
    output reg [3:0] Q
);

always @(posedge clk or posedge reset) begin
    if (reset)
        Q <= 4'b0001;
    else
        Q <= {Q[2:0], Q[3]};
end

endmodule
```

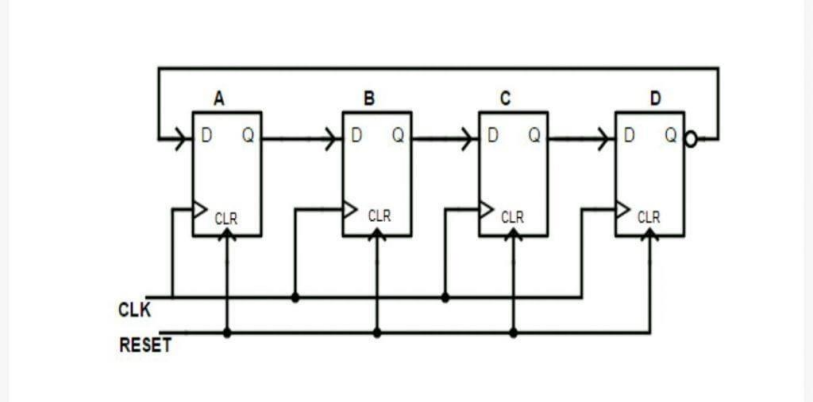
Output:



Experiment 7: Johnson Counter

Aim: To design a 4-bit Johnson counter using four flip-flops.

Logic diagram:



Code:

```
// Code your testbench here


module tb_johnson_counter;

reg clk, reset;
wire [3:0] Q;

johnson_counter dut(
    .clk(clk),
    .reset(reset),
    .Q(Q)
);

always #5 clk = ~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_johnson_counter);

    clk = 0;
    reset = 1;
    #10;

    reset = 0;
    repeat (12) #10;

    $finish;
end

endmodule
```

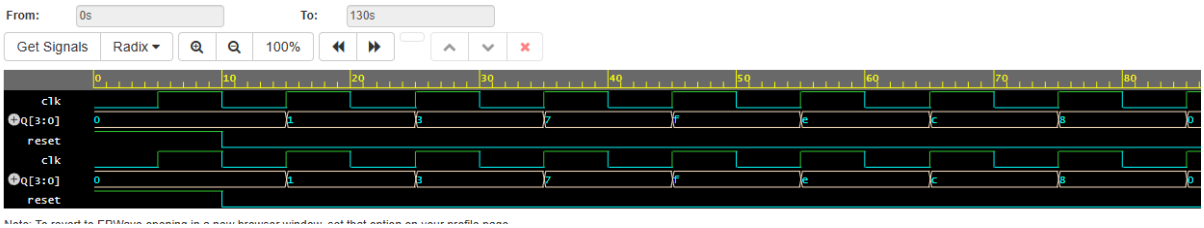
```
// Code your design here


module johnson_counter(
    input clk,
    input reset,
    output reg [3:0] Q
);

always @(posedge clk or posedge reset) begin
    if (reset)
        Q <= 4'b0000;
    else
        Q <= {Q[2:0], ~Q[3]};
end

endmodule
```

Output:

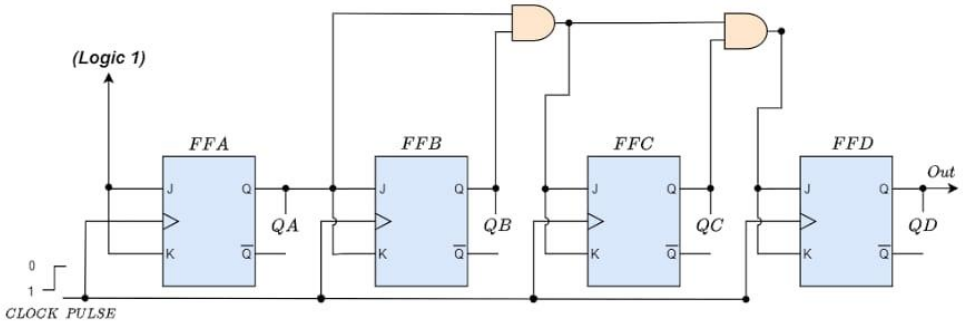


Experiment 8: Synchronous and Asynchronous counter

A. Synchronous counter

Aim: To design a 4-bit synchronous binary up-counter in which all flip-flops are triggered by the same clock.

Logic diagram:



Code:

```
// Code your testbench here

module tb_sync_counter;

reg clk, reset;
wire [3:0] Q;

sync_counter dut(
    .clk(clk),
    .reset(reset),
    .Q(Q)
);

always #5 clk = ~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_sync_counter);

    clk = 0;
    reset = 1;
    #10;

    reset = 0;
    repeat (20) #10;

    $finish;
end

endmodule
```

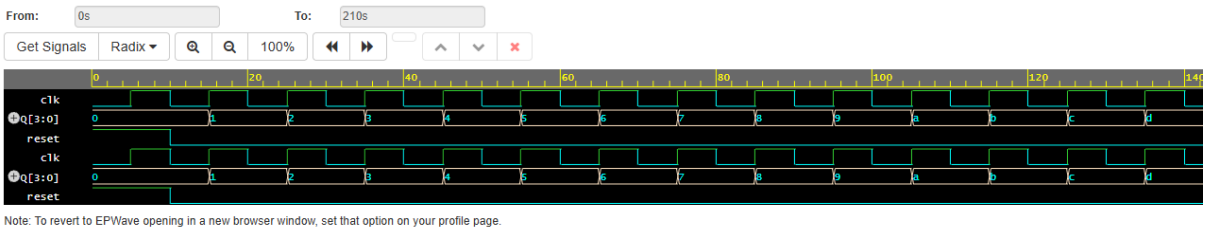
```
// Code your design here

module sync_counter(
    input clk,
    input reset,
    output reg [3:0] Q
);

always @(posedge clk or posedge reset) begin
    if (reset)
        Q <= 4'b0000;
    else
        Q <= Q + 1'b1;
end

endmodule
```

Output:

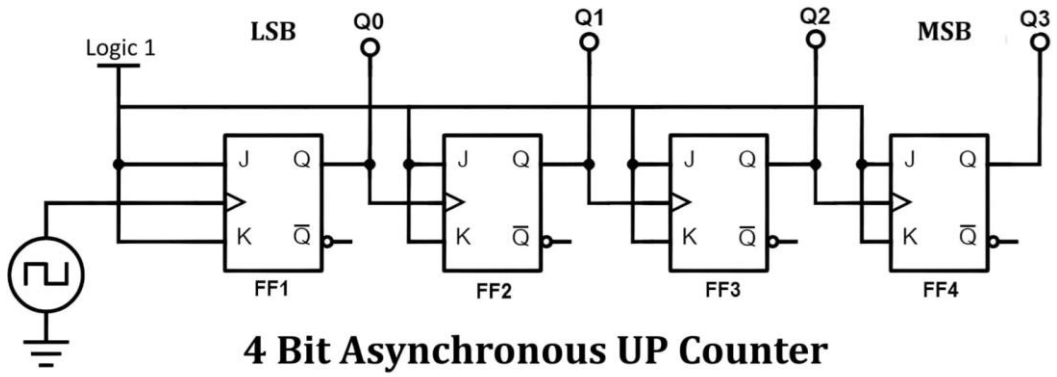


Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

B.Asynchronous Counter

Aim: To design a 4-bit ripple counter in which each successive flip-flop is clocked by the preceding stage.

Logic diagram:



Code:

```
// Code your testbench here


module tb_async_counter;

reg clk, reset;
wire [3:0] Q;

async_counter dut(
    .clk(clk),
    .reset(reset),
    .Q(Q)
);

always #5 clk = ~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_async_counter);

    clk = 0;
    reset = 1;
    #12;

    reset = 0;
    repeat (20) #10;

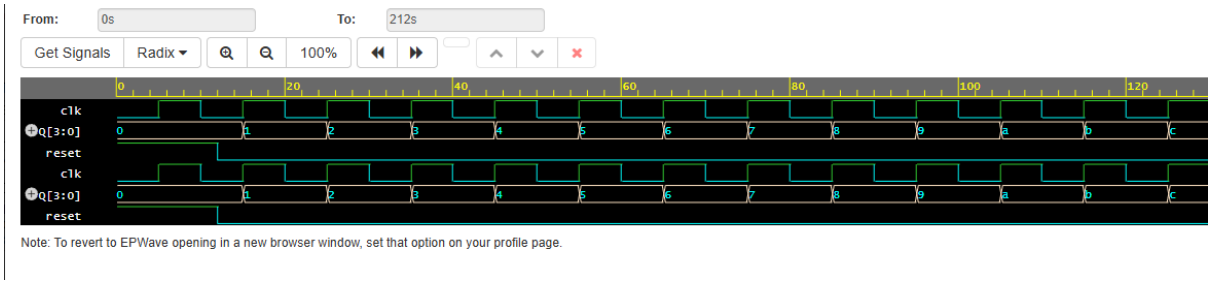
    $finish;
end

endmodule
```

```
// Code your design here
module async_counter(
    input clk,
    input reset,
    output reg [3:0] Q
);

always @(posedge clk or posedge reset) begin
    if (reset)
        Q[0] <= 0;
    else
        Q[0] <= ~Q[0];
end
always @(negedge Q[0] or posedge reset) begin
    if (reset)
        Q[1] <= 0;
    else
        Q[1] <= ~Q[1];
end
always @(negedge Q[1] or posedge reset) begin
    if (reset)
        Q[2] <= 0;
    else
        Q[2] <= ~Q[2];
end
always @(negedge Q[2] or posedge reset) begin
    if (reset)
        Q[3] <= 0;
    else
        Q[3] <= ~Q[3];
end
endmodule
```

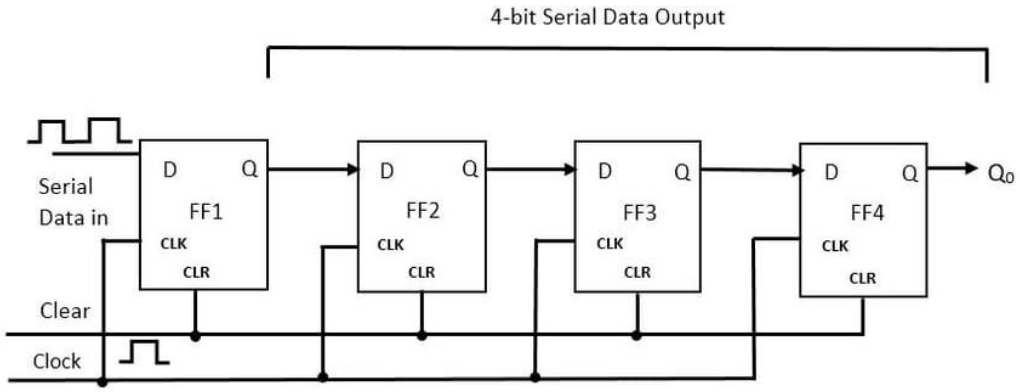
Output:



Experiment 9: Shift Register

Aim: To design a 4-bit serial-in, serial-out (SISO) shift register.

Logic diagram:



Code:

```
// Code your testbench here

module tb_shift_register;

reg clk, reset, serial_in;
wire [3:0] Q;
wire serial_out;

shift_register dut(
    .clk(clk),
    .reset(reset),
    .serial_in(serial_in),
    .Q(Q),
    .serial_out(serial_out)
);

always #5 clk = ~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_shift_register);

    clk = 0;
    reset = 1;
    serial_in = 0;
    #10;

    reset = 0;
    serial_in = 1; #10;
    serial_in = 0; #10;
    serial_in = 1; #10;
    serial_in = 1; #10;
    serial_in = 0; #10;

    $finish;
end

endmodule
```

```
// Code your design here

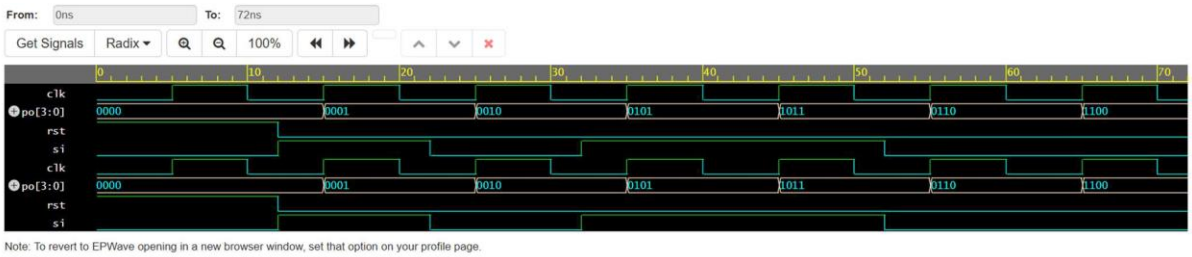

module shift_register(
    input clk,
    input reset,
    input serial_in,
    output reg [3:0] Q,
    output serial_out
);

assign serial_out = Q[3];

always @(posedge clk or posedge reset) begin
    if (reset)
        Q <= 4'b0000;
    else
        Q <= {Q[2:0], serial_in};
end

endmodule
```

Output:

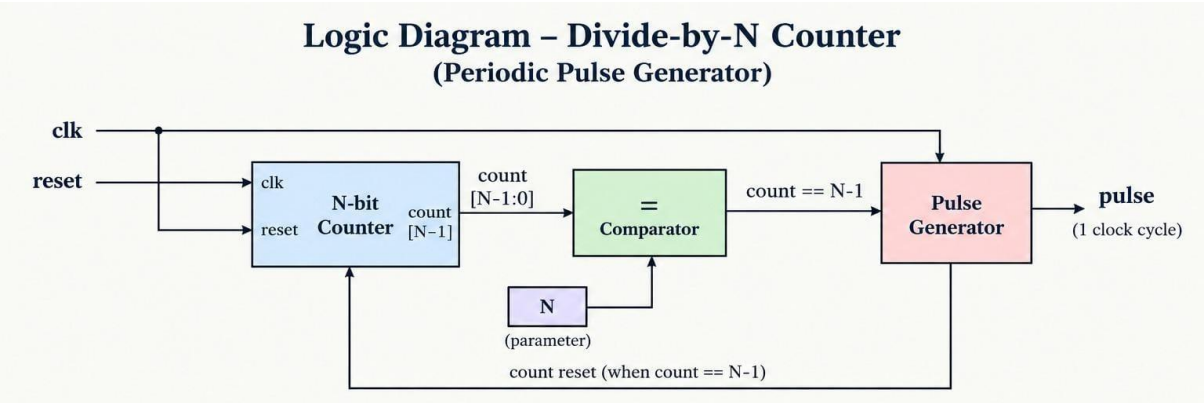


Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

Experiment 10: Divide-by-N Counter

Aim: To design a counter that generates a periodic output pulse after every N clock cycles.

Logic diagram:



Code:

```
// Code your testbench here
module tb_shift_register;
reg clk, reset, serial_in;
wire [3:0] Q;
wire serial_out;
shift_register dut(
    .clk(clk),
    .reset(reset),
    .serial_in(serial_in),
    .Q(Q),
    .serial_out(serial_out)
);
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_shift_register);
    clk = 0;
    reset = 1;
    serial_in = 0;
    #10;
    reset = 0;
    serial_in = 1; #10;
    serial_in = 0; #10;
    serial_in = 1; #10;
    serial_in = 1; #10;
    serial_in = 0; #10;

    $finish;
end

endmodule
```

```
// Code your design here

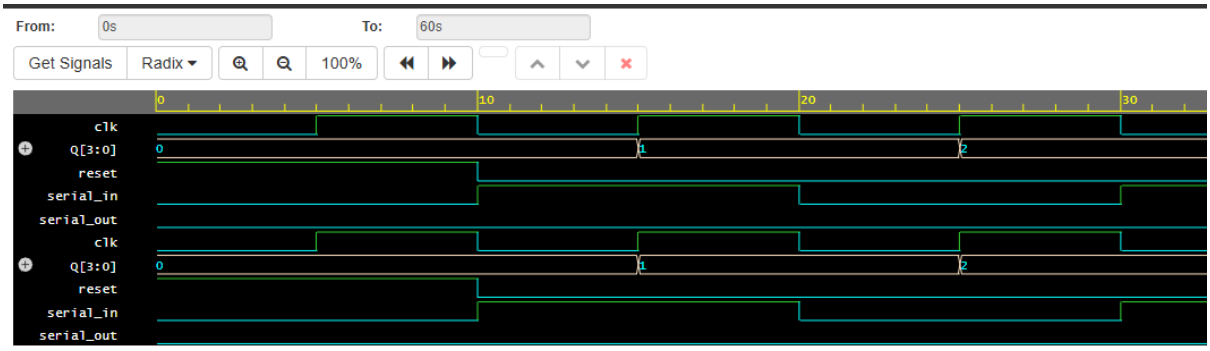
module shift_register(
    input clk,
    input reset,
    input serial_in,
    output reg [3:0] Q,
    output serial_out
);

assign serial_out = Q[3];

always @(posedge clk or posedge reset) begin
    if (reset)
        Q <= 4'b0000;
    else
        Q <= {Q[2:0], serial_in};
end

endmodule
```

Output:

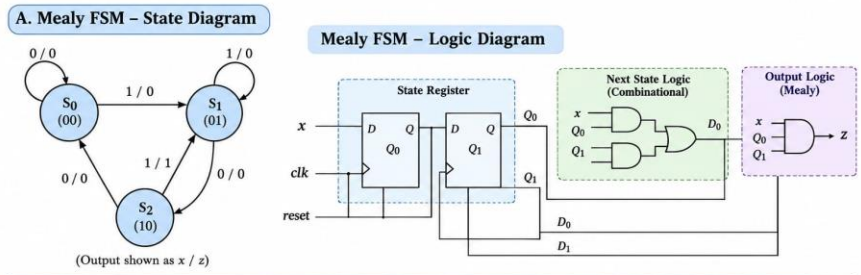


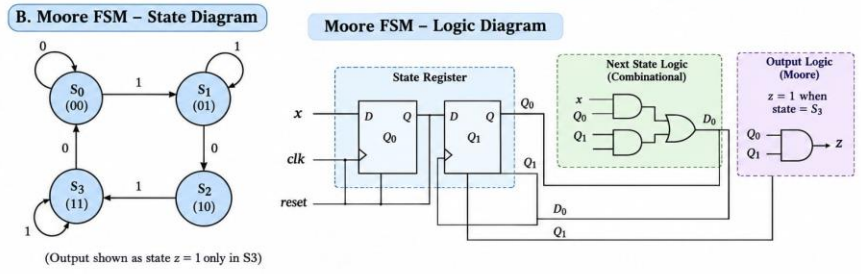
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

Experiment 11: Mealy and Moore Finite State Machines (FSMs)

Aim: To implement Mealy and Moore FSMs for detecting the binary sequence 101.

Logic diagram:





Code:

A. Mealy FSM

```
module tb_moore_101_detector;
  reg       clk;
  reg       rst;
  reg       in;
  reg       out;
  moore_101_detector uut (
    .clk(clk),
    .rst(rst),
    .in(in),
    .out(out)
  );
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_moore_101_detector);
    clk = 1'b0;
    rst = 1'b1;
    in = 1'b0;
    #12
    rst = 1'b0;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    $finish;
  end
  initial begin
    $monitor("Time=%0t | rst=%b, in=%b | out=%b",
$time, rst, in, out);
  end
endmodule
```

```
module moore_101_detector (
    input  reg       clk,
    input  reg       rst,
    input  reg       in,
    output reg       out
);

  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
  reg [1:0] state, next_state;

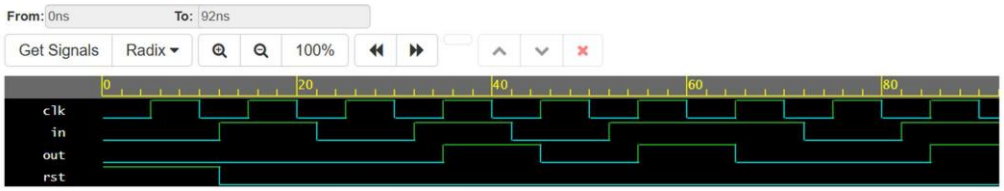
  always @(posedge clk or posedge rst) begin
    if (rst)
      state <= S0;
    else
      state <= next_state;
  end

  always @(*) begin
    case (state)
      S0: next_state = in ? S1 : S0;
      S1: next_state = in ? S1 : S2;
      S2: next_state = in ? S3 : S0;
      S3: next_state = in ? S1 : S2;
      default: next_state = S0;
    endcase
  end

  always @(*) begin
    out = (state == S3);
  end

endmodule
```

Output:



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

B. Moore FSM

```
module tb_moore_101_detector;
  reg       clk;
  reg       rst;
  reg       in;
  reg       out;
  moore_101_detector uut (
    .clk(clk),
    .rst(rst),
    .in(in),
    .out(out)
  );
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_moore_101_detector);
    clk = 1'b0;
    rst = 1'b1;
    in = 1'b0;
    #12;
    rst = 1'b0;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    $finish;
  end
  initial begin
    $monitor("Time=%0t | rst=%b, in=%b | out=%b",
$time, rst, in, out);
  end
endmodule
```

```
module moore_101_detector (
    input  reg       clk,
    input  reg       rst,
    input  reg       in,
    output reg       out
);

  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
  reg [1:0] state, next_state;

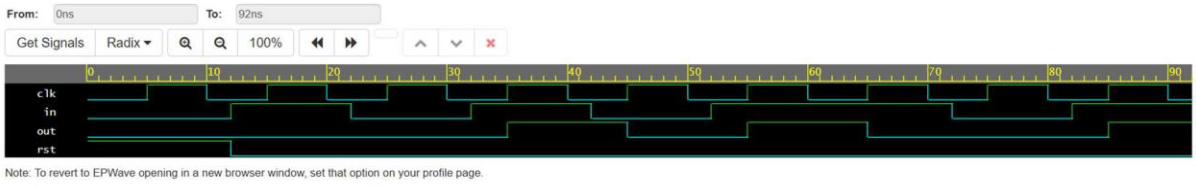
  always @(posedge clk or posedge rst) begin
    if (rst)
      state <= S0;
    else
      state <= next_state;
  end

  always @(*) begin
    case (state)
      S0: next_state = in ? S1 : S0;
      S1: next_state = in ? S1 : S2;
      S2: next_state = in ? S3 : S0;
      S3: next_state = in ? S1 : S2;
      default: next_state = S0;
    endcase
  end

  always @(*) begin
    out = (state == S3);
  end

endmodule
```

Output:

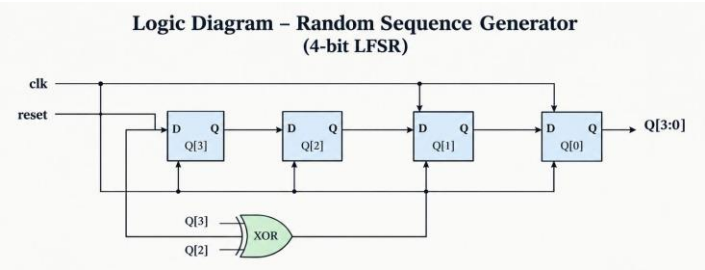


Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

Experiment 12: Random Sequence Generator

Aim: To design a pseudo-random sequence generator using a linear feedback shift register (LFSR).

Logic diagram:



Code:

```
module tb_random_seq_generator;

  reg       clk;
  reg       rst;
  reg [3:0] lfsr_out;

  random_seq_generator uut (
    .clk(clk),
    .rst(rst),
    .lfsr_out(lfsr_out)
  );

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_random_seq_generator);

    clk = 1'b0;
    rst = 1'b1;
    #12;

    rst = 1'b0;
    #160;

    $finish;
  end

  initial begin
    $monitor("Time=%0t | rst=%b | lfsr_out=%b",
$time, rst, lfsr_out);
  end

endmodule
```

```
module random_seq_generator (
    input  reg       clk,
    input  reg       rst,
    output reg [3:0] lfsr_out
);

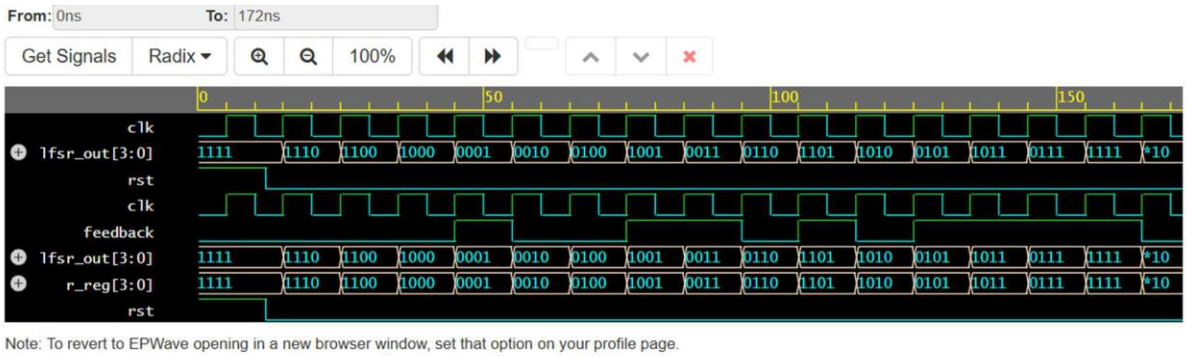
  reg [3:0] r_reg;
  reg       feedback;

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      r_reg <= 4'b1111;
    end else begin
      feedback = r_reg[3] ^ r_reg[2];
      r_reg <= {r_reg[2:0], feedback};
    end
  end

  always @(*) begin
    lfsr_out = r_reg;
  end

endmodule
```

Output:



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.